Phase 1: Core Tensor and memory layout

What I built

A `Tensor` class that stores its numbers once, in a flat 1D buffer called `Storage`, and describes how to read that buffer as a grid using three fields: `shape`, `strides` and `offset`. Slicing, transposing, permuting, squeezing, unsqueezing and expanding all create **views**: new Tensor objects that point at the same storage with different shape, strides or offset. No numbers are copied. The only operation that copies is `contiguous()` (and `reshape()` when it has to call it).

Files: `tensorforge/tensor.py`, `tests/test_views.py` (42 tests, checked against NumPy).

Why it matters

Every array library (NumPy, PyTorch, JAX) works this way. It's why PyTorch can transpose a huge tensor instantly, and why some operations fail with "tensor is not contiguous." Understanding storage, strides and views is what lets you debug shape errors, avoid accidental copies, and reason about memory and speed.

How it works

The position formula

Element (i_0, i_1, ...) lives at this position in storage:

```
pos = offset + i₀ × stride₀ + i₁ × stride₁ + …
```

Contiguous strides

For a normal row-major tensor, the last stride is 1, and each earlier stride is the next stride times the next size.

Shape (2, 3, 4) → strides (12, 4, 1).

Worked example

```
t = tensor([[0, 1, 2], [3, 4, 5]])
```

- Storage: [0, 1, 2, 3, 4, 5], shape (2, 3), strides (3, 1), offset 0.
- Element (1, 2): pos = 0 + 1×3 + 2×1 = 5 → storage[5] = 5. ✓

`t.T` (transpose):

- Shape (3, 2), strides (1, 3), offset 0. Same storage.
- `t.T[1, 0]` : pos = 0 + 1×1 + 0×3 = 1 → storage[1] = 1. ✓ (In the original that's t[0, 1] = 1.)

How each view changes the fields

Operation	What changes
`t[i]` (integer)	offset += i × stride; that dimension is removed
`t[a:b:s]` (slice)	offset += a × stride; stride × s; size = number of elements in the range
`t[None]`	new dimension of size 1
`transpose` / `permute` / `.T`	shape and strides reordered together
`unsqueeze` / `squeeze`	add or remove size-1 dimensions
`expand`	size-1 dims grow, with stride 0 (same element repeated)
`view`	new shape with contiguous strides, same storage (needs a contiguous tensor)
`reshape`	`contiguous()` first if needed, then `view`

Contiguity

A tensor is contiguous when its strides match the normal row-major strides for its shape (size-1 dimensions are ignored). Slicing off rows keeps it contiguous; transposing or stepping through columns does not.

Key code

- `_contiguous_strides(shape)` : computes row-major strides by walking backwards through the shape.
- `__getitem__` : normalizes the key (handles `...` and missing dims), then walks each part, updating offset, shape and strides. Returns a new Tensor on the same storage.

- `is_contiguous()` / `contiguous()` : check the strides; copy into fresh storage in row-major order only when needed.
- `__setitem__` : finds the storage positions of the selected elements and writes into them, so every view sharing that storage sees the change. It also bumps `storage.version` , which Phase 3 will use to detect in-place changes that break gradients.

Bugs I hit and how I fixed them

- **Pasted code into the wrong places.** Placeholder lines ended up after a `return` , where VS Code dimmed them as unreachable. Fix: replaced the whole file with a known-good version, checked it with `wc -l` and `grep` , and closed the old editor tab without saving.
- **`ImportError: cannot import name 'rand'` .** The test file was updated but `tensor.py` wasn't saved. Lesson: when an import fails, check that the file on disk actually contains the name (`grep -n "def rand" tensorforge/tensor.py`).
- **pytest showed fewer tests than expected.** Usually an unsaved file or a test pasted in the wrong place. `pytest -v` lists every test by name.
- **Committed `.DS_Store` .** Added it to `.gitignore` and removed it with `git rm --cached .DS_Store` .

Interview questions

Why is a transpose O(1) but `contiguous()` O(n)?
Transpose only swaps two numbers in `shape` and `strides` ; no data moves, so the cost doesn't depend on the tensor's size. `contiguous()` has to visit every element and copy it into new storage in row-major order, so it costs time proportional to the number of elements, n.

What's the difference between `view` and `reshape` ?
`view` always shares storage and never copies, so it only works if the tensor is contiguous; otherwise it raises an error. `reshape` gives the same result but copies first when the tensor isn't contiguous. With `view` you know no copy happened; with `reshape` you don't.

Why can iterating over a transposed array be slower than over the original?
Memory is read in chunks (cache lines). Walking a contiguous tensor reads neighboring positions, so each chunk is fully used. Walking a transposed tensor jumps by large strides, so each step may load a new chunk and waste most of it, causing more cache misses.

What I'd still like to understand better

- (Fill in yourself: anything that still feels unclear.)